

React JS Assignment

Topic 1: React Lifecycle

Q1. What are React lifecycle methods?

A: React lifecycle methods are special functions that run at different stages of a component's life like when it is created, updated, or removed.

Q2. What are the three main phases of the React lifecycle?

A: The three phases are Mounting (component is created), Updating (component changes), and Unmounting (component is removed).

Q3. What happens in the Mounting phase?

A: In Mounting, React creates the component and puts it on the screen. Functions like `constructor()` and `componentDidMount()` run here.

Q4. What happens in the Updating phase?

A: Updating happens when props or state changes. React re-renders the component to show new data.

Q5. What is the use of `componentDidMount()`?

A: It runs after the component first shows up on the screen. It is often used to fetch data from an API.

Q6. What is the use of `componentWillUnmount()`?

A: It runs when a component is removed. It is used to clean things like timers or event listeners.

Topic 2: Routing

Q7. What is React Router?

A: React Router is a library that lets us move between pages in a React app without reloading the whole page.

Q8. What is the difference between `BrowserRouter` and `HashRouter`?

A: `BrowserRouter` uses clean URLs like `/home`, while `HashRouter` uses a `#` in the URL like `/#/home`.

Q9. What is a `<Link>` in React Router?

A: `<Link>` is used instead of `<a>` to move between pages without refreshing.

Q10. What is the difference between Route and Switch?

A: Route defines which component should show for a path, Switch makes sure only the first matching route is shown.

Q11. What is useNavigate in React Router v6?

A: `useNavigate` is a hook used to move to another page programmatically (by code).

Q12. How do we pass parameters in React Router?

A: We can pass dynamic values like `/user/:id` and read them using `useParams()`.

Topic 3: State Management

Q13. What is state in React?

A: State is a built-in object that stores data for a component. When state changes, the component updates automatically.

Q14. What is the difference between state and props?

A: State belongs to the component itself, props are passed from parent to child.

Q15. What is useState in React?

A: `useState` is a hook to create and update state in functional components.

Q16. What is useReducer in React?

A: `useReducer` is another hook for managing complex state using actions and reducers.

Q17. What is Context API in React?

A: Context API allows us to share state across many components without passing props manually.

Q18. What is the difference between Context API and Redux?

A: Context API is simple and built-in for smaller apps, Redux is more powerful for big apps with many states.

Topic 4: Forms in React

Q19. How are forms handled in React?

A: Forms in React are controlled using state. Input values are stored in state and updated on change.

Q20. What is a controlled component?

A: A controlled component is an input field that gets its value from React state.

Q21. What is an uncontrolled component?

A: An uncontrolled component is an input that works on its own, and React does not control its value directly.

Q22. How do we handle multiple inputs in React forms?

A: By using one handler function and checking `event.target.name` to update the correct state.

Q23. How do we handle form submission in React?

A: By using `onSubmit` event and preventing page reload with `event.preventDefault()`.

Q24. How do we validate form input in React?

A: We can check input values with conditions before submitting or use libraries like Formik.

Topic 5: React Authentication

Q25. How is authentication done in React?

A: Authentication in React is usually done using tokens or sessions, and state is used to track logged-in users.

Q26. What is JWT in authentication?

A: JWT (JSON Web Token) is a secure way to share user data between client and server.

Q27. How do we store authentication tokens in React?

A: Tokens can be stored in `localStorage` or cookies.

Q28. How do we protect routes in React?

A: By creating `PrivateRoute` components that check if a user is logged in before showing a page.

Q29. What is the difference between authentication and authorization?

A: Authentication checks identity (who you are), authorization checks permission (what you can do).

Q30. How do we log out a user in React?

A: By removing the token from `localStorage` or cookies and clearing user state.

MongoDB (20 Questions)

Topic 1: CRUD OPERATION

Q1. What does CRUD mean in MongoDB?

A: CRUD means **Create, Read, Update, and Delete**. These are the four basic actions we can perform on data in MongoDB.

Q2. How do we insert data in MongoDB?

A: We can use **insertOne()** for a single document and **insertMany()** for multiple documents inside a collection.

Q3. How do we read or find data in MongoDB?

A: We use the **find()** method to get documents. Example:

```
db.users.find({name: "Ali"})
```

Q4. How do we update data in MongoDB?

A: We use **updateOne()** for a single document or **updateMany()** to update multiple documents.

Q5. How do we delete data in MongoDB?

A: We use **deleteOne()** to remove one document and **deleteMany()** to remove multiple documents.

Topic 2: Querying

Q6. What is querying in MongoDB?

A: Querying means searching data from a collection using filters like conditions, ranges, or patterns.

Q7. How do we query with conditions in MongoDB?

A: We can use operators like **\$gt** (greater than), **\$lt** (less than), **\$eq** (equal). Example:

```
db.users.find({age: {$gt: 20}})
```

Q8. How do we search with multiple conditions in MongoDB?

A: We use **\$and** and **\$or**. Example:

```
db.users.find({$or: [{age: 20}, {name: "Ali"}]})
```

Q9. How do we search with patterns in MongoDB?

A: We can use **Regular Expressions**. Example:

```
db.users.find({name: /Ali/})
```

Q10. How do we sort and limit results in MongoDB?

A: We use **sort()** and **limit()**. Example:

```
db.users.find().sort({age: 1}).limit(5)
```

Topic 3: Integration With Node.js

Q11. How do we connect MongoDB with Node.js?

A: We use the official **mongodb** package or **mongoose**. First, install it with npm, then connect using **MongoClient.connect()**.

Q12. What is Mongoose in Node.js?

A: Mongoose is an **ODM (Object Data Modeling)** library for MongoDB. It makes working with schemas and models easier.

Q13. How do we insert data from Node.js into MongoDB?

A: After connecting, we can use:

```
db.collection("users").insertOne({name: "Ali"})
```

Q14. How do we read data from MongoDB in Node.js?

A: We use **find()** with **toArray()**. Example:

```
const users = await db.collection("users").find().toArray();
```

Q15. Why is MongoDB popular with Node.js?

A: Because both use **JavaScript**, they work smoothly together, making development faster and easier.

Topic 4: Indexing In MongoDB

Q16. What is indexing in MongoDB?

A: Indexing makes searches **faster**. It works like an index in a book.

Q17. How do we create an index in MongoDB?

A: We use **createIndex()**. Example:

```
db.users.createIndex({name: 1})
```

Q18. What is the difference between single field and compound index?

A: Single field index is on one field (e.g., name), compound index is on multiple fields (e.g., name and age).

Q19. What is a unique index in MongoDB?

A: A unique index makes sure no **duplicate values** are stored in that field. Example: unique **email IDs**.

Q20. What is the disadvantage of too many indexes?

A: Too many indexes can **slow down writes** because MongoDB has to update all indexes whenever data changes.